LABORATORIO GUIADO

Consumo de un JSON público como API simulada con Angular

Catálogo de productos · HttpClient + Observable + Bootstrap

Institución	**Universidad Tecnológica del Perú (UTP)**
Curso / período	JavaScript Avanzado · 2026-II
Unidad / semana	Unidad 2 · Semana 9 · Repaso previo a evaluación
Docente	Ing. Mg. Carlos Alberto Ponte Ramírez
Modalidad	Laboratorio de cómputo

1. Situación motivadora

Una tienda de tecnología publica un catálogo de 15 productos en un archivo productos.json alojado en GitHub Pages. Angular debe solicitar esos datos mediante HTTP GET, mostrarlos en una tabla y permitir búsquedas y filtros. El archivo JSON simula el comportamiento de un endpoint de consulta de una API REST: no existe un backend de Spring Boot ni una base de datos detrás de esta dirección.

Pregunta detonadora: ¿qué partes de Angular son iguales al consumir un JSON remoto y una API GET de Spring Boot?

2. Logro del laboratorio

Al finalizar, el estudiante configura HttpClient, crea un servicio, obtiene un arreglo de productos mediante Observable y subscribe(), lo presenta en Bootstrap e implementa búsqueda por nombre y filtro de stock insuficiente.

- Consumir un JSON publicado en Internet mediante HTTP GET.
- Definir una interfaz TypeScript con id, nombre, precio, stock y estadoStock.
- Mostrar los 15 productos y verificar la solicitud en la pestaña Network.
- Buscar por nombre, filtrar por estado y recargar el listado.

3. Fuente de datos (API simulada)

Método: GET | Archivo: productos.json | Alojamiento: GitHub Pages. Dirección indicada para el laboratorio (verificar en el navegador antes de la clase):

```
https://careli2021.github.io/api_examen/api/productos.json
```

Es una respuesta JSON estática. Permite consultar (GET), pero NO permite insertar, editar ni eliminar productos por POST, PUT o DELETE. El botón «Recargar» vuelve a solicitar el mismo archivo, cuyos datos no cambian salvo que se publique una nueva versión.

```
[
  {"id":1,"nombre":"O - Laptop Lenovo","precio":6800,
   "stock":98,"estadoStock":"OK"},
  {"id":2,"nombre":"R - Mouse Logitech","precio":80,
   "stock":30,"estadoStock":"OK"},
  {"id":3,"nombre":"L - Teclado Mecánico","precio":180,
   "stock":2,"estadoStock":"INSUFICIENTE"}
]
```

El archivo remoto completo incluye 15 registros. Regla usada: stock menor que 5 → INSUFICIENTE; stock de 5 o más → OK.

4. Preparación del proyecto

- Utilizar el proyecto Angular standalone desarrollado en las sesiones previas.
- Comprobar que Bootstrap está configurado y que hay conexión a Internet.
- Si ya existe provideHttpClient() en app.config.ts, no duplicarlo.
- Crear servicio y componente con Angular CLI, si todavía no existen.

```
npm install -g @angular/cli
ng new repaso
cd repaso
ng serve -o

ng generate service services/producto
ng generate component productos
```

5. Configurar HttpClient — app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient()
  ]
};
```

6. Servicio — src/app/services/producto.ts

Nota: Angular CLI puede generar producto.ts o producto.service.ts según la versión/configuración. Usa el nombre real y ajusta las importaciones.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Producto {
  id: number;
  nombre: string;
  precio: number;
  stock: number;
  estadoStock: string;
}

@Injectable({ providedIn: 'root' })
export class ProductoService {
  private url = 'https://careli2021.github.io/api_examen/api/productos.json';
  constructor(private http: HttpClient) {}
  obtenerProductos(): Observable<Producto[]> {
    return this.http.get<Producto[]>(this.url);
  }
}
```

7. Componente — src/app/productos/productos.ts

El componente pide datos una sola vez al comenzar y almacena el resultado. Los filtros se aplican localmente en Angular, sin consultar al servidor en cada pulsación. Al recibir la respuesta de HttpClient, ChangeDetectorRef.detectChanges() fuerza la actualización de la tabla y del indicador de carga (útil en configuraciones zoneless).

```
import { Component, OnInit, ChangeDetectorRef } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Producto, ProductoService } from '../services/producto';

@Component({
  selector: 'app-productos',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './productos.html'
})
```

```
export class Productos implements OnInit {
  productos: Producto[] = [];
  busqueda = '';
  soloInsuficientes = false;
  cargando = false;
  error = '';

  constructor(
    private servicio: ProductoService,
    private cdr: ChangeDetectorRef
  ) {}
  ngOnInit(): void { this.cargarProductos(); }

  cargarProductos(): void {
    this.cargando = true;
    this.error = '';
    this.cdr.detectChanges();
    this.servicio.obtenerProductos().subscribe({
      next: (datos: Producto[]) => {
        this.productos = datos;
        this.cargando = false;
        this.cdr.detectChanges();
      },
      error: err => {
        console.error('Error HTTP:', err);
        this.error = 'No se pudieron obtener los productos.';
        this.cargando = false;
        this.cdr.detectChanges();
      }
    });
  }

  get productosFiltrados(): Producto[] {
    const texto = this.busqueda.trim().toLowerCase();
    return this.productos.filter(p =>
      p.nombre.toLowerCase().includes(texto) &&
      (!this.soloInsuficientes || p.estadoStock === 'INSUFICIENTE')
    );
  }
}
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Producto, ProductoService } from '../services/producto';

@Component({
  selector: 'app-productos',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './productos.html'
})
export class Productos implements OnInit {
  productos: Producto[] = [];
  busqueda = '';
  soloInsuficientes = false;
  cargando = false;
  error = '';

  constructor(private servicio: ProductoService) {}
  ngOnInit(): void { this.cargarProductos(); }

  cargarProductos(): void {
    this.cargando = true;
    this.error = '';
    this.servicio.obtenerProductos().subscribe({
      next: datos => { this.productos = datos; this.cargando = false; },
      error: err => {
        console.error('Error HTTP:', err);
        this.error = 'No se pudieron obtener los productos.';
        this.cargando = false;
      }
    });
```

```
  }

  get productosFiltrados(): Producto[] {
    const texto = this.busqueda.trim().toLowerCase();
    return this.productos.filter(p =>
      p.nombre.toLowerCase().includes(texto) &&
      (!this.soloInsuficientes || p.estadoStock === 'INSUFICIENTE')
    );
  }
}
```

8. Vista — src/app/productos/productos.html

```
<div class="container py-4">
  <h2 class="mb-2">Catálogo de productos</h2>
  <p class="text-muted">Consulta de JSON público con Angular HttpClient</p>

  <div class="row g-2 mb-3">
    <div class="col-md-6">
      <input class="form-control" placeholder="Buscar por nombre"
             [(ngModel)]="busqueda">
    </div>
    <div class="col-md-4 form-check pt-2">
      <input id="ins" type="checkbox" class="form-check-input"
             [(ngModel)]="soloInsuficientes">
      <label for="ins" class="form-check-label">Solo stock insuficiente</label>
    </div>
    <div class="col-md-2">
      <button class="btn btn-primary w-100" (click)="cargarProductos()">
        Recargar
      </button>
    </div>
  </div>

  <p *ngIf="cargando">Consultando productos...</p>
  <div *ngIf="error" class="alert alert-danger">{{ error }}</div>
  <div class="table-responsive" *ngIf="!cargando && !error">
    <table class="table table-striped table-hover">
      <thead class="table-dark"><tr>
        <th>ID</th><th>Nombre</th><th>Precio</th>
        <th>Stock</th><th>Estado</th>
      </tr></thead>
      <tbody>
        <tr *ngFor="let p of productosFiltrados">
          <td>{{ p.id }}</td><td>{{ p.nombre }}</td>
          <td>{{ p.precio | currency:'PEN':'symbol':'1.2-2' }}</td>
          <td>{{ p.stock }}</td>
          <td><span class="badge"
                [ngClass]="p.estadoStock === 'OK' ? 'bg-success' : 'bg-danger'">
            {{ p.estadoStock }}
          </span></td>
        </tr>
      </tbody>
    </table>
    <p>Resultados: {{ productosFiltrados.length }}</p>
    <p *ngIf="productosFiltrados.length === 0" class="text-muted">
      No se encontraron productos.
    </p>
  </div>
</div>
```

9. Registrar ruta — app.routes.ts

Si el proyecto ya tiene otras rutas, conserva sus definiciones y añade la ruta productos. Ejemplo mínimo:

```
import { Routes } from '@angular/router';
import { Productos } from './productos/productos';

export const routes: Routes = [
  { path: 'productos', component: Productos },
```

```
  { path: '', redirectTo: 'productos', pathMatch: 'full' }
];
```

10. Ejecución, diagnóstico y verificación

```
ng serve -o
```

- Abrir /productos: deben visualizarse 15 registros al inicio.
- Buscar «Mouse» y verificar que la tabla muestre solo los nombres coincidentes.
- Activar «Solo stock insuficiente» y confirmar que los resultados se reduzcan.
- Pulsar Recargar y comprobar que el archivo JSON sigue respondiendo.
- Abrir F12 → Network → Fetch/XHR → productos.json y comprobar HTTP 200 y Response con arreglo JSON.

Si en el navegador la URL JSON funciona pero Angular no muestra datos, revisar Console y Network. Un error CORS se observa allí; no se resuelve cambiando el código de filtrado. Si la solicitud devuelve una página HTML o 404, revisar la URL publicada en GitHub Pages. Si Network devuelve HTTP 200 y el JSON correcto, pero permanece «Cargando», comprobar que el callback next actualice cargando=false y ejecute this.cdr.detectChanges(); también hacerlo en error para reflejar el mensaje.

11. Comparación: JSON publicado y API REST de Spring Boot

Característica	JSON en GitHub Pages	API Spring Boot
Lectura GET	Sí	Sí
Respuesta	Arreglo JSON fijo	JSON generado por backend
POST / PUT / DELETE	No	Posible con endpoints
MySQL / Docker	No necesita	Según aplicación
Uso en este laboratorio	Consulta y filtros en Angular	Fuera del alcance

12. Evidencias a entregar

- Captura de app.config.ts y provideHttpClient().
- Captura del servicio con HttpClient.get<Producto[]>().
- Captura de la tabla completa y de la búsqueda por nombre.
- Captura del filtro de stock insuficiente.
- Captura de Network con estado 200 y respuesta JSON.

13. Reto adicional (opcional)

- Añadir filtro por precio mínimo y máximo en el navegador.
- Mostrar el total de unidades en stock y el valor del inventario.
- Agregar ordenamiento por precio ascendente y descendente.

14. Rúbrica de evaluación — 20 puntos

Criterio	Puntaje
Configura HttpClient correctamente	4
Servicio + interfaz + GET remoto	4
Observable, subscribe y control de errores	4
Tabla, búsqueda y filtro de stock	4
Evidencia de ejecución y Network	4
TOTAL	20

15. Síntesis

El patrón Componente → Servicio → HttpClient → Observable → subscribe() sigue siendo válido. Aquí productos.json funciona como una API de solo lectura simulada. En una API REST real, las operaciones CRUD requieren métodos y lógica adicionales del servidor. En este proyecto se utiliza ChangeDetectorRef para reflejar los datos asíncronos en pantalla.

Referencia: Angular HttpClient — https://angular.dev/guide/http | GitHub Pages — https://docs.github.com/pages